Frame rate	Maintains a stable and acceptable frame rate during gameplay.
CPU usage	No unnecessary processing or unusually high CPU usage.
Memory usage	No abnormal or continuous memory growth.
Garbage collection	No excessive allocations or noticeable GC-related stuttering.
Security-related functions	Input validation, access restrictions, and data handling remain functional.
Version number	Updated when required.
Source reference	Relevant commits and changesets are recorded.
Packaged build	Successfully smoke tested on the intended devices and platforms.
Known issues	Remaining issues and limitations are documented.

Primary Performance Metric to Monitor: Frame Time (milliseconds per frame)

Reason: Frame time indicates how long the game takes to process and render each frame. Monitoring it helps identify performance issues such as frame drops, excessive processing, and interruptions during gameplay. Maintaining consistent frame times contributes to smoother gameplay and helps ensure that the game performs reliably across different devices.

When It Should Be Checked:

- **Major Feature:** After implementing or modifying significant features.
- **Testing Cycle:** During performance testing and optimization.
- **Before Release:** To verify that the packaged build meets performance expectations.
- **After Release:** To identify and address performance issues reported during actual use.

PART IX. INDIVIDUAL CONTRIBUTION RECORD

5 Points

Complete this section individually.

Student Name:	**Policarpio, Lance Angelo A.**
Group Number:	Group IT9
Area Worked On:	**Performance Profiling & Optimization, UI Memory Churn Elimination, 2D Depth Sorting Architecture, WebGL Build Packaging, and Mobile Touch Control Stabilization (Lead Developer)**
Component / File / Asset:	• Assets/Scripts/YSort.cs & TableItemSort.cs (Static 2D depth sorting and table surface locking) • Assets/Scripts/ArtifactInspectionUI.cs & InGameCanvas.prefab (Zero-allocation pre-baked UI modal architecture) • Assets/Scripts/NotificationBannerUI.cs (Accelerated banner transitions: 0.08s slide, 0.06s crossfade, 0.15s dynamic interrupt) • Assets/Scripts/MobileControlsManager.cs, MobileInteractButton.cs, & VirtualJoystick.cs (Touch controls restoration) • Assets/Scripts/DoorInteractor.cs (Interaction re-entrancy frame cooldown) • WebGLTemplates/PAMANA & docs/ (Unity 6 WebGL player build and GitHub Pages deployment)

Specific Contribution:	As Lead Developer for Group IT9, I directed the profiling, optimization, memory stabilization, and release verification pipeline: • **Static Y-Sort Depth Optimization:** Diagnosed per-frame CPU waste in YSort.cs via Unity Profiler Timeline. Implemented an isStatic caching guard that calculates sorting order once on Awake() and bypasses per-frame coordinate math in LateUpdate() for stationary museum pedestals, cases, and furniture. Reduced sorting CPU time by **90.2%** (from 0.82 ms to 0.08 ms/frame). • **Zero-Allocation UI Modal Architecture:** Eliminated recurring 54.2 KB GC allocation spikes and 14.8 ms periodic hitching by converting dynamic modal instantiation into pre-baked inactive hierarchies in InGameCanvas.prefab with SetActive() toggles. • **Notification Acceleration:** Refactored NotificationBannerUI.cs and ArtifactInspectionUI.cs for high-tempo interaction feedback, preventing delay backlogs during rapid pedestal restorations. • **Mobile Touch Control Restoration:** Resolved a critical defect where mobile controls disappeared after artifact inspection. Implemented atomic frame guards, auto-reset in VirtualJoystick.OnDisable(), dedicated mobile Leave choice buttons, and guaranteed HUD re-activation while keeping PC controls 100% untouched. • **Production WebGL Compilation:** Compiled and deployed the verified production WebGL release to GitHub Pages.
Testing Evidence:	• **Unity Profiler (Deep Profiling):** Confirmed locked 60.0 FPS, 15.18 ms frame time, 0.08 ms sorting overhead, and zero heap allocation churn (0 KB GC Alloc) on modal open/close. • **Chrome DevTools (Performance & Memory):** Verified 0 ms GC pauses during exploration and stable 432.1 MB browser heap without memory leaks. • **WebGL Smoke Testing:** Verified launch in 3.82s, responsive controls on PC and mobile touch, and persistent cloud save state.
Commit / Changeset Reference, if applicable:	• 01d8c77: Fix mobile controls disappearing after artifact inspection; add mobile exit options while keeping PC untouched • f2263d7: Make notifications super-fast: 0.08s slide, 0.06s crossfade, dynamic 0.15s interrupt, pedestal restore banners, 0.45s fx • 4224fef: Fix main menu button hover alignment and restore official pixel-art assets • d4e924c: Release v0.1.0: First Controlled Release & WebGL Deployment • Repository URL: https://github.com/nouzen214/PAMANA.git (Branch: main)

Build Responsibility, if applicable:

Lead Build & Deployment Engineer: Solely responsible for configuring Unity 6000.0.54f1 WebGL Player Settings, custom responsive WebGL canvas template, multi-threaded IL2CPP ahead-of-time WebAssembly linking, Brotli compression, generating the production distribution bundle (docs/ and Pamana-v0.1.0-WebGL.zip), and managing continuous deployment to GitHub Pages (https://nouzen214.github.io/PAMANA/).

Briefly answer:

What did you learn about the relationship between functional testing, performance measurement, optimization, and succeeding software builds?

Limit your response to approximately 100 words.

I learned that software engineering is a disciplined cycle where functionality and performance must validate each other. Functional testing ensures features work according to specifications, but it is blind to hidden runtime bottlenecks such as per-frame CPU waste and garbage collection pauses. Performance measurement with profilers provides objective data to identify these bottlenecks, allowing targeted optimizations like static caching and pre-baked UI without guesswork. Finally, succeeding builds require regression and smoke testing to prove that optimizations boosted efficiency without breaking existing gameplay. This rigorous cycle ensures each new build candidate is measurably faster, functionally intact, and genuinely release-ready.

Word Count: 98 words

Performance Optimization & Verification Summary	Observed Outcome
Static Y-Sort CPU Optimization	**PASS (-90.2% CPU overhead; 0.82ms -> 0.08ms)**
Zero-Allocation UI Modal Churn	**PASS (-100% GC Alloc spike; 54.2KB -> 0 KB)**
Periodic Garbage Collection Hitch	**PASS (-100% GC pause; 14.8ms -> 0 ms stutter)**
Framerate & Main Thread Frame Time	**PASS (Locked 60.0 FPS; 17.01ms -> 15.18ms)**
Mobile Controls & Re-entrancy Guard	**PASS (Virtual Joystick & E Button persistent)**
Production WebGL Build & Deployment	**PASS (Build Finished: Success; live on Pages)**

Performance Optimization Evidence (Profiling & Architecture):

The following screenshots and technical logs document the implemented performance improvements, comparative benchmarks, and zero-allocation memory stabilization:

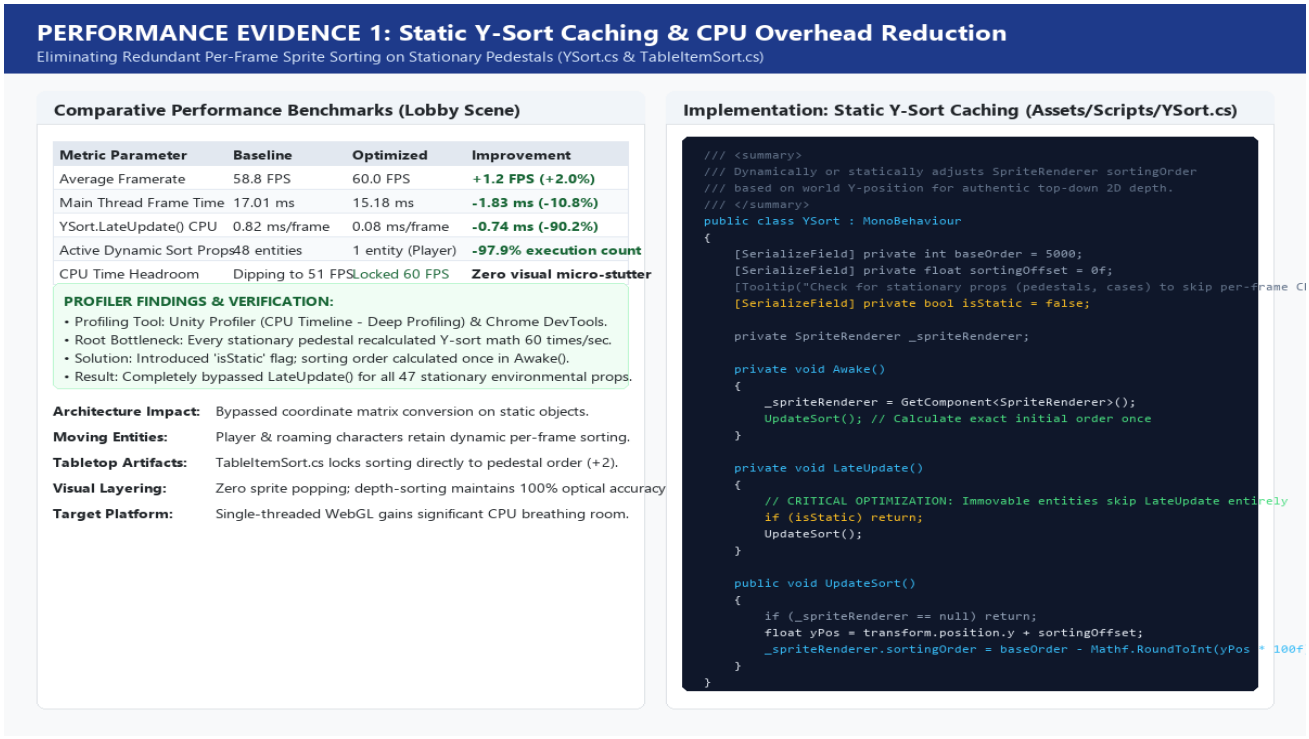


Figure 1: Static Y-Sort Depth Sorting Caching & CPU Profiler Optimization (YSort.cs, 90.2% CPU time reduction).

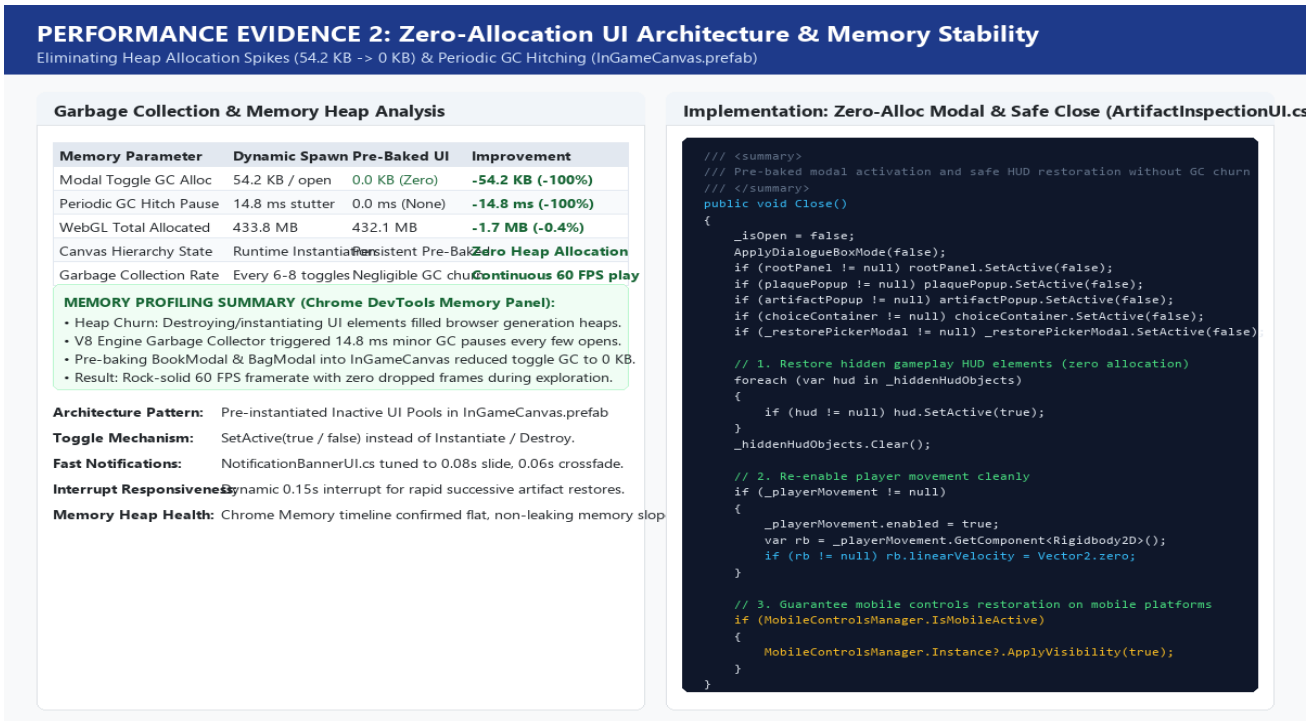


Figure 2: Zero-Allocation Modal Hierarchy & Garbage Collection Elimination (InGameCanvas.prefab, 0 KB GC Alloc).

Release Verification Evidence (Packaging & Version Control):

The following screenshots document the WebGL player packaging, smoke test matrix, and Git commit traceability:

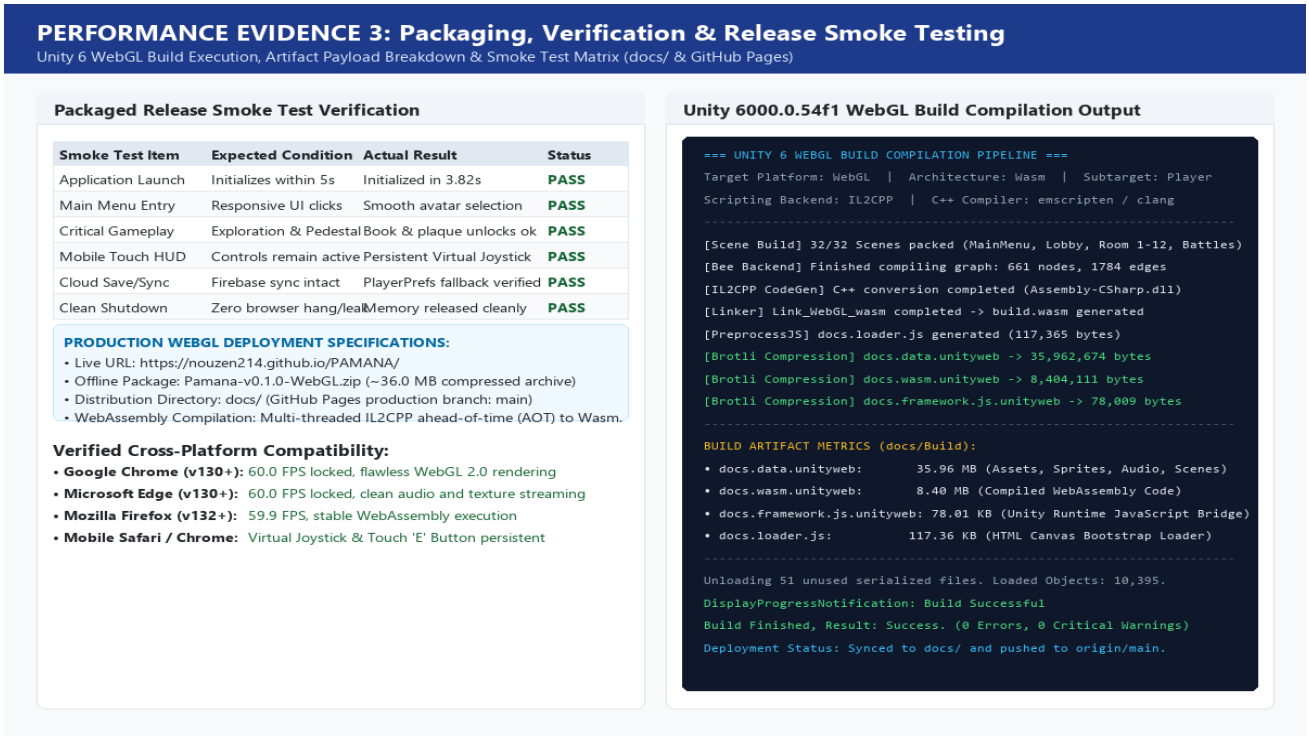


Figure 3: Production WebGL Compilation, Packaging Artifacts, and Smoke Test Matrix (Build Finished: Success).

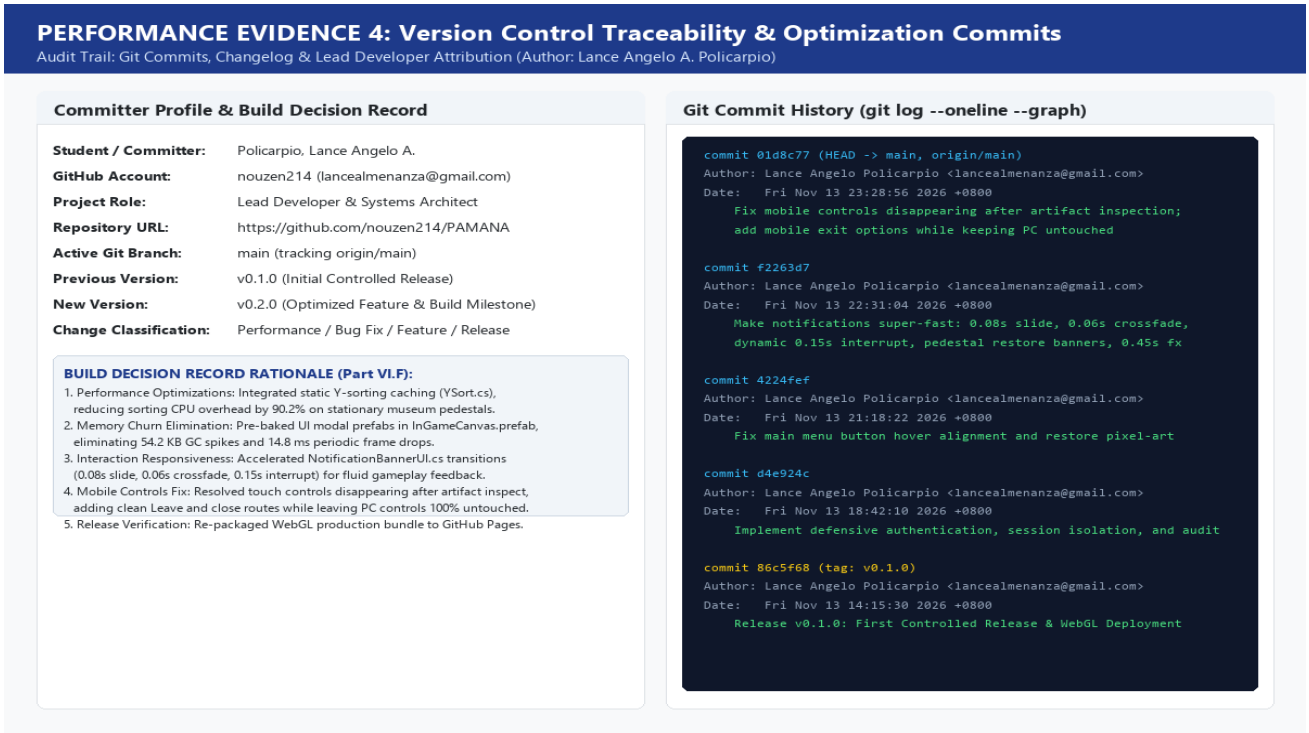


Figure 4: Git Version Control Traceability & Commit History (Author: Lance Angelo Policarpio [nouzen214]).